

EXPERIMENT – 6

Name: K ANSHU

Roll No: 2520030440

Section: 8

Title

FIFO Client-Server Communication and Signal Handling

Aim

To implement a FIFO-based client-server communication system and handle SIGINT, SIGTERM, and SIGUSR1 signals.

Theory

- A **FIFO (First In First Out)** is a named pipe used for communication between processes.
- `mkfifo()` creates a named FIFO.
- The server opens the FIFO and waits for messages from the client.
- The client writes a message into the FIFO.
- SIGUSR1 is used to demonstrate user-defined signal handling.
- SIGINT and SIGTERM are signals used to request process termination.

Algorithm

1. Start the program.
2. Register handlers for SIGUSR1, SIGINT, and SIGTERM.
3. Create a FIFO using `mkfifo()`.
4. Display the server PID.
5. Open the FIFO for reading.
6. Read the message sent by the client.
7. Display the received message.
8. Handle SIGUSR1, SIGINT, and SIGTERM when received.
9. Close the FIFO.
10. Terminate the server.

Program Code

```
k_anusri@DESKTOP-TM20BAK × + v
k_anusri@DESKTOP-TM20BAQ:~$
k_anusri@DESKTOP-TM20BAQ:~$ cat > exp6_fifo.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <signal.h>
#include <string.h>

#define FIFO_NAME "ossp_fifo"

void handle_signal(int sig)
{
    if (sig == SIGUSR1)
        printf("SIGUSR1 received.\n");
    else if (sig == SIGINT)
        printf("SIGINT received. Server terminating.\n");
    else if (sig == SIGTERM)
        printf("SIGTERM received. Server terminating.\n");
}

int main()
{
    int fd;
    char buffer[100];

    signal(SIGUSR1, handle_signal);
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);

    mkfifo(FIFO_NAME, 0666);

    printf("FIFO Server started.\n");
    printf("PID: %d\n", getpid());

    while (1)
    {
        fd = open(FIFO_NAME, O_RDONLY);

        if (fd == -1)
        {
            perror("open");
            return 1;
        }

        int n = read(fd, buffer, sizeof(buffer) - 1);

        if (n > 0)
        {
            buffer[n] = '\0';
            printf("Message received: %s\n", buffer);
        }

        close(fd);
    }

    unlink(FIFO_NAME);

    return 0;
}
EOF
```

Execution and Screenshots

Screenshot 1 — Compilation and Server Execution

```
k_anusri@DESKTOP-TM20BAQ:~$ gcc exp6_fifo.c -o exp6_fifo
k_anusri@DESKTOP-TM20BAQ:~$ ./exp6_fifo
FIFO Server started.
```

Screenshot 2 — FIFO Client Communication

IN TERMINAL 2

```
k_anusri@DESKTOP-TM20BAQ:~$ echo "Hello from FIFO client." > ossp_fifo
```

IN TERMINAL 1

```
Message received: Hello from FIFO client.
```

Screenshot 3 — SIGUSR1

In Terminal 2

```
k_anusri@DESKTOP-TM20BAQ:~$ kill -SIGUSR1 1234
-bash: kill: (1234) - No such process
```

In Terminal 1

```
k_anusri@DESKTOP-TM20BAQ:~$ ./exp6_fifo
FIFO Server started.
PID: 1695
```

In Terminal 2

Screenshot 4 — SIGINT / SIGTERM

In Terminal 1

```
k_anusri@DESKTOP-TM20BAQ:~$ gcc exp6_fifo.c -o exp6_fifo
k_anusri@DESKTOP-TM20BAQ:~$ ./exp6_fifo
FIFO Server started.
PID: 1695
Message received: Hello from FIFO client.

./exp6_fifo
```

```
k_anusri@DESKTOP-TM20BAQ:~$ kill -SIGTERM 1456
-bash: kill: (1456) - No such process
```

Result

FIFO-based client-server communication was implemented successfully, and the required SIGINT, SIGTERM, and SIGUSR1 signals were handled.